

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import warnings
warnings.filterwarnings('ignore')

DATASET1 = '../data/train'
DATASET2 = '../data/val'
DATASET3 = '../data/test'

print("Imports loaded.")
print("Dataset 1:", os.path.exists(DATASET1))
print("Dataset 2:", os.path.exists(DATASET2))
print("Dataset 3:", os.path.exists(DATASET3))
```

```
Imports loaded.
Dataset 1: True
Dataset 2: True
Dataset 3: True
```

Tech Stack Summary

Purpose	Library
Image loading & processing	Pillow, opencv-python
Data manipulation	numpy, pandas
Preprocessing & scaling	sklearn.preprocessing
Dimensionality reduction	sklearn.decomposition.PCA
ML models	sklearn (LogisticRegression, RandomForest, SVM, KNN)
Hyperparameter tuning	GridSearchCV, RandomizedSearchCV
Evaluation metrics	sklearn.metrics
Visualization	matplotlib, seaborn
Model saving	joblib
Bonus deep learning	tensorflow.keras
Bonus SOM	minisom
Notebook & export	jupyter, nbconvert

```
In [2]: def explore_dataset(path, name):
print(f"\n--- {name} ---")
```

```

total = 0
for label in ['NORMAL', 'PNEUMONIA']:
    folder = os.path.join(path, label)
    files = [f for f in os.listdir(folder)
              if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    print(f" {label}: {len(files)} images")
    total += len(files)
print(f" TOTAL: {total} images")

explore_dataset(DATASET1, 'Dataset 1 Train')
explore_dataset(DATASET2, 'Dataset 2 Val')
explore_dataset(DATASET3, 'Dataset 3 Test')

```

```

--- Dataset 1 Train ---
NORMAL: 1341 images
PNEUMONIA: 3875 images
TOTAL: 5216 images

```

```

--- Dataset 2 Val ---
NORMAL: 8 images
PNEUMONIA: 8 images
TOTAL: 16 images

```

```

--- Dataset 3 Test ---
NORMAL: 234 images
PNEUMONIA: 390 images
TOTAL: 624 images

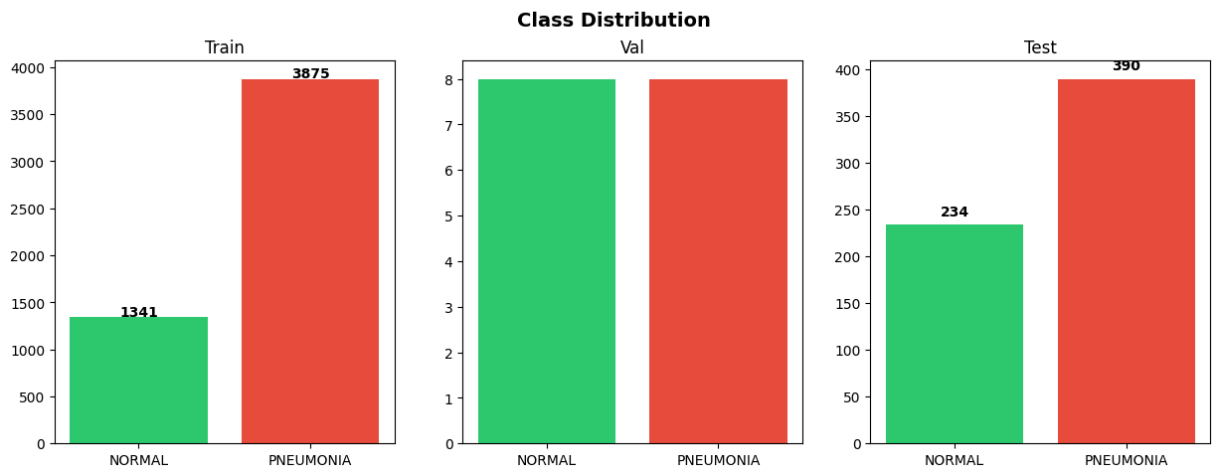
```

```

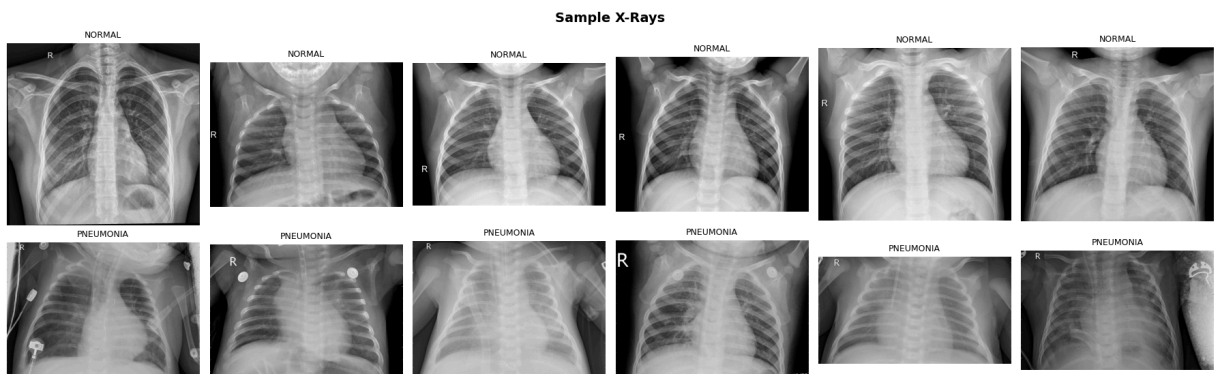
In [3]: datasets = {
        'Train': DATASET1,
        'Val': DATASET2,
        'Test': DATASET3
    }

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, (name, path) in zip(axes, datasets.items()):
    counts = []
    for label in ['NORMAL', 'PNEUMONIA']:
        folder = os.path.join(path, label)
        files = [f for f in os.listdir(folder)
                  if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        counts.append(len(files))
    ax.bar(['NORMAL', 'PNEUMONIA'], counts, color=['#2ecc71', '#e74c3c'])
    ax.set_title(name)
    for i, v in enumerate(counts):
        ax.text(i, v+10, str(v), ha='center', fontweight='bold')
plt.suptitle('Class Distribution', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../outputs/figures/class_distribution.png', dpi=150)
plt.show()

```



```
In [4]: fig, axes = plt.subplots(2, 6, figsize=(18, 6))
        for col, label in enumerate(['NORMAL', 'PNEUMONIA']):
            folder = os.path.join(DATASET1, label)
            files = [f for f in os.listdir(folder)
                     if f.lower().endswith(('.jpg', '.jpeg', '.png'))][:6]
            for row, fname in enumerate(files):
                img = Image.open(os.path.join(folder, fname)).convert('L')
                axes[col][row].imshow(img, cmap='gray')
                axes[col][row].set_title(label, fontsize=9)
                axes[col][row].axis('off')
        plt.suptitle('Sample X-Rays', fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.savefig('../outputs/figures/sample_xrays.png', dpi=150)
        plt.show()
```



```
In [5]: def load_dataset(path, size=(128, 128), rotate=False):
        X, y = [], []
```

```

for label, val in [('NORMAL', 0), ('PNEUMONIA', 1)]:
    folder = os.path.join(path, label)
    files = [f for f in os.listdir(folder)
              if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    for fname in files:
        try:
            img = Image.open(os.path.join(folder, fname)).convert('L')
            img = img.resize(size)
            X.append(np.array(img).flatten() / 255.0)
            y.append(val)
            if rotate:
                img_rotated = img.rotate(15)
                X.append(np.array(img_rotated).flatten() / 255.0)
                y.append(val)
        except:
            print(f"Skipped: {fname}")
    return np.array(X), np.array(y)

print("load_dataset function ready!")

```

load_dataset function ready!

```

In [6]: print("Loading Dataset 1 (Train)...")
X_train_raw, y_train = load_dataset(DATASET1, rotate=False)
print(f"Loaded training data: {X_train_raw.shape}")

print("\nLoading Dataset 2 (Val)...")
X_val_raw, y_val = load_dataset(DATASET2)
print(f"Loaded validation data: {X_val_raw.shape}")

print("\nLoading Dataset 3 (Test)...")
X_test_raw, y_test = load_dataset(DATASET3)
print(f"Loaded test data: {X_test_raw.shape}")

```

Loading Dataset 1 (Train)...
Loaded training data: (5216, 16384)

Loading Dataset 2 (Val)...
Loaded validation data: (16, 16384)

Loading Dataset 3 (Test)...
Loaded test data: (624, 16384)

```

In [7]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_val_scaled = scaler.transform(X_val_raw)
X_test_scaled = scaler.transform(X_test_raw)
print("Scaling is complete.")

pca = PCA(n_components=100, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_val_pca = pca.transform(X_val_scaled)
X_test_pca = pca.transform(X_test_scaled)
print("PCA is complete.")
print(f"Variance explained: {pca.explained_variance_ratio_.sum():.2%}")

```

Scaling is complete.

PCA is complete.
Variance explained: 87.88%

```
In [8]: print("=" * 50)
print("Preprocessing is finished, and the data is ready for modeling.")
print("=" * 50)
print(f"X_train_pca : {X_train_pca.shape}")
print(f"X_val_pca   : {X_val_pca.shape}")
print(f"X_test_pca  : {X_test_pca.shape}")
print(f"y_train     : Normal={(y_train==0).sum()} Pneumonia={(y_train==1).sum()}")
```

=====

Preprocessing is finished, and the data is ready for modeling.

=====

```
X_train_pca : (5216, 100)
X_val_pca   : (16, 100)
X_test_pca  : (624, 100)
y_train     : Normal=1341 Pneumonia=3875
```

```
In [9]: import joblib

# Cache directory
CACHE_DIR = '../data/preprocessed'
os.makedirs(CACHE_DIR, exist_ok=True)

# Save raw arrays
np.save(f'{CACHE_DIR}/X_train.npy', X_train_raw)
np.save(f'{CACHE_DIR}/y_train.npy', y_train)
np.save(f'{CACHE_DIR}/X_val.npy', X_val_raw)
np.save(f'{CACHE_DIR}/y_val.npy', y_val)
np.save(f'{CACHE_DIR}/X_test.npy', X_test_raw)
np.save(f'{CACHE_DIR}/y_test.npy', y_test)

# Save scaler and PCA
joblib.dump(scaler, f'{CACHE_DIR}/scaler.joblib')
joblib.dump(pca, f'{CACHE_DIR}/pca.joblib')

print("Cached preprocessed arrays and transforms.")
print(f"Saved under: {CACHE_DIR}")
```

Cached preprocessed arrays and transforms.
Saved under: ../data/preprocessed

```
In [10]: from sklearn.model_selection import train_test_split

# Split 1: straightforward 80/20 train/test split
X_split1_train, X_split1_test, y_split1_train, y_split1_test = train_test_split(
    X_train_pca, y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

print("Split 1 is ready: 80/20 train/test split complete.")
print(f"Train : {X_split1_train.shape}")
print(f"Test  : {X_split1_test.shape}")
```

Split 1 is ready: 80/20 train/test split complete.

Train : (4172, 100)

Test : (1044, 100)

```
In [11]: # Split 2: use the provided train/validation/test folders
# The dataset folders already give us this split.
X_split2_train = X_train_pca
X_split2_val = X_val_pca
X_split2_test = X_test_pca
y_split2_train = y_train
y_split2_val = y_val
y_split2_test = y_test

print("Split 2 is ready: train, validation, and test sets are loaded.")
print(f" Train : {X_split2_train.shape}")
print(f" Val : {X_split2_val.shape}")
print(f" Test : {X_split2_test.shape}")
```

Split 2 is ready: train, validation, and test sets are loaded.

Train : (5216, 100)

Val : (16, 100)

Test : (624, 100)

```
In [12]: from sklearn.model_selection import StratifiedKFold

# Split 3: 5-fold stratified cross-validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("Split 3 is ready: 5-fold stratified cross-validation is set up.")
print(f" Folds : 5")
print(f" Total images : {X_train_pca.shape[0]}")
print(f" Each fold train: ~{int(X_train_pca.shape[0]*0.8)} images")
print(f" Each fold test : ~{int(X_train_pca.shape[0]*0.2)} images")
```

Split 3 is ready: 5-fold stratified cross-validation is set up.

Folds : 5

Total images : 5216

Each fold train: ~4172 images

Each fold test : ~1043 images

```
In [13]: from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, PredefinedSplit, cross_val
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score,
                             confusion_matrix, RocCurveDisplay,
                             ConfusionMatrixDisplay)

import matplotlib.pyplot as plt

print("Modeling libraries are ready.")
```

Modeling libraries are ready.

```
In [14]: # Logistic Regression – Split 1 (Simple Train/Test)
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_split1_train, y_split1_train)
```

```

y_pred = lr_model.predict(X_split1_test)
y_prob = lr_model.predict_proba(X_split1_test)[: ,1]

print("LR – Split 1 (Simple Train/Test)")
print(f" Accuracy : {accuracy_score(y_split1_test, y_pred):.4f}")
print(f" Precision : {precision_score(y_split1_test, y_pred):.4f}")
print(f" Recall : {recall_score(y_split1_test, y_pred):.4f}")
print(f" F1 : {f1_score(y_split1_test, y_pred):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split1_test, y_prob):.4f}")

```

```

LR – Split 1 (Simple Train/Test)
Accuracy : 0.9617
Precision : 0.9742
Recall : 0.9742
F1 : 0.9742
ROC-AUC : 0.9922

```

```

In [15]: # Logistic Regression – Split 2 (Train/Val/Test)
lr_model2 = LogisticRegression(max_iter=1000, random_state=42)
lr_model2.fit(X_split2_train, y_split2_train)

y_pred2 = lr_model2.predict(X_split2_test)
y_prob2 = lr_model2.predict_proba(X_split2_test)[: ,1]

print("LR – Split 2 (Train/Val/Test)")
print(f" Accuracy : {accuracy_score(y_split2_test, y_pred2):.4f}")
print(f" Precision : {precision_score(y_split2_test, y_pred2):.4f}")
print(f" Recall : {recall_score(y_split2_test, y_pred2):.4f}")
print(f" F1 : {f1_score(y_split2_test, y_pred2):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split2_test, y_prob2):.4f}")

```

```

LR – Split 2 (Train/Val/Test)
Accuracy : 0.7452
Precision : 0.7143
Recall : 0.9872
F1 : 0.8288
ROC-AUC : 0.8951

```

```

In [16]: # Logistic Regression – Split 3 (K-Fold CV)
lr_model3 = LogisticRegression(max_iter=1000, random_state=42)

cv_results = cross_validate(
    lr_model3, X_train_pca, y_train,
    cv=skf,
    scoring=['accuracy', 'f1', 'roc_auc'],
    return_train_score=True
)

print("LR – Split 3 (K-Fold CV)")
print(f" Accuracy : {cv_results['test_accuracy'].mean():.4f}")
print(f" F1 : {cv_results['test_f1'].mean():.4f}")
print(f" ROC-AUC : {cv_results['test_roc_auc'].mean():.4f}")

```

```

LR – Split 3 (K-Fold CV)
Accuracy : 0.9565
F1 : 0.9708
ROC-AUC : 0.9886

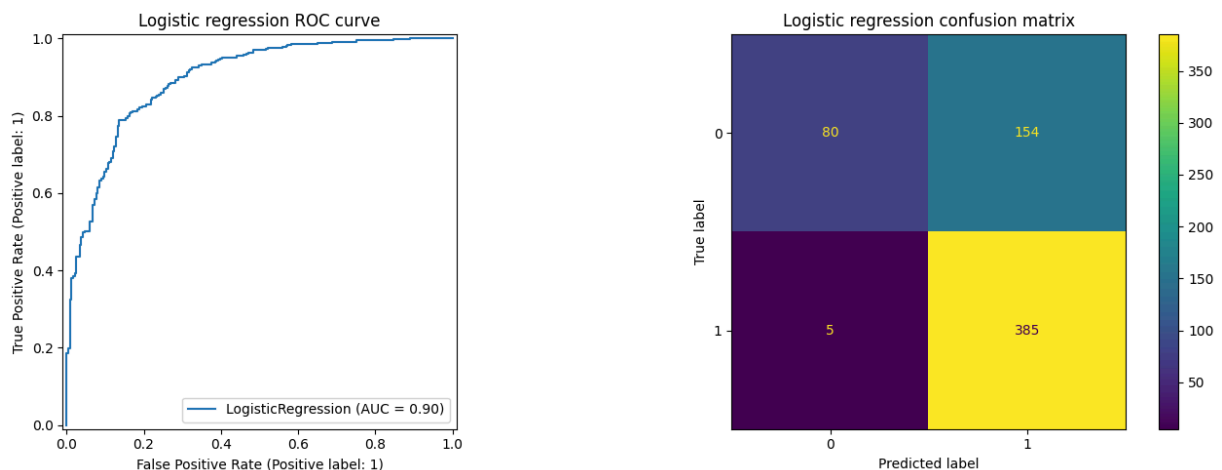
```

```
In [17]: # ROC Curve + Confusion Matrix for LR
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

RocCurveDisplay.from_estimator(
    lr_model2, X_split2_test, y_split2_test, ax=axes[0])
axes[0].set_title('Logistic regression ROC curve')

ConfusionMatrixDisplay.from_estimator(
    lr_model2, X_split2_test, y_split2_test, ax=axes[1])
axes[1].set_title('Logistic regression confusion matrix')

plt.tight_layout()
plt.savefig('../outputs/figures/LR_evaluation.png', dpi=150)
plt.show()
print("Logistic regression plots saved.")
```



Logistic regression plots saved.

```
In [18]: from sklearn.ensemble import RandomForestClassifier

print("Random Forest is ready.")
```

Random Forest is ready.

```
In [19]: # Random Forest – Split 1 (Simple Train/Test)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_split1_train, y_split1_train)

y_pred_rf = rf_model.predict(X_split1_test)
y_prob_rf = rf_model.predict_proba(X_split1_test)[:,1]

print("RF – Split 1 (Simple Train/Test)")
print(f" Accuracy : {accuracy_score(y_split1_test, y_pred_rf):.4f}")
print(f" Precision : {precision_score(y_split1_test, y_pred_rf):.4f}")
print(f" Recall : {recall_score(y_split1_test, y_pred_rf):.4f}")
print(f" F1 : {f1_score(y_split1_test, y_pred_rf):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split1_test, y_prob_rf):.4f}")
```

RF – Split 1 (Simple Train/Test)

Accuracy : 0.9387
Precision : 0.9363
Recall : 0.9845
F1 : 0.9598
ROC-AUC : 0.9865

```
In [20]: # Random Forest – Split 2 (Train/Val/Test)
rf_model2 = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model2.fit(X_split2_train, y_split2_train)

y_pred_rf2 = rf_model2.predict(X_split2_test)
y_prob_rf2 = rf_model2.predict_proba(X_split2_test)[:,1]

print("RF – Split 2 (Train/Val/Test)")
print(f" Accuracy : {accuracy_score(y_split2_test, y_pred_rf2):.4f}")
print(f" Precision : {precision_score(y_split2_test, y_pred_rf2):.4f}")
print(f" Recall : {recall_score(y_split2_test, y_pred_rf2):.4f}")
print(f" F1 : {f1_score(y_split2_test, y_pred_rf2):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split2_test, y_prob_rf2):.4f}")
```

RF – Split 2 (Train/Val/Test)

Accuracy : 0.7500
Precision : 0.7159
Recall : 0.9949
F1 : 0.8326
ROC-AUC : 0.9159

```
In [21]: # Random Forest – Split 3 (K-Fold CV)
rf_model3 = RandomForestClassifier(n_estimators=100, random_state=42)

cv_results_rf = cross_validate(
    rf_model3, X_train_pca, y_train,
    cv=skf,
    scoring=['accuracy', 'f1', 'roc_auc'],
    return_train_score=True
)

print("RF – Split 3 (K-Fold CV)")
print(f" Accuracy : {cv_results_rf['test_accuracy'].mean():.4f}")
print(f" F1 : {cv_results_rf['test_f1'].mean():.4f}")
print(f" ROC-AUC : {cv_results_rf['test_roc_auc'].mean():.4f}")
```

RF – Split 3 (K-Fold CV)

Accuracy : 0.9337
F1 : 0.9566
ROC-AUC : 0.9829

```
In [22]: # ROC Curve + Confusion Matrix for RF
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

RocCurveDisplay.from_estimator(
    rf_model2, X_split2_test, y_split2_test, ax=axes[0])
axes[0].set_title('Random forest ROC curve')

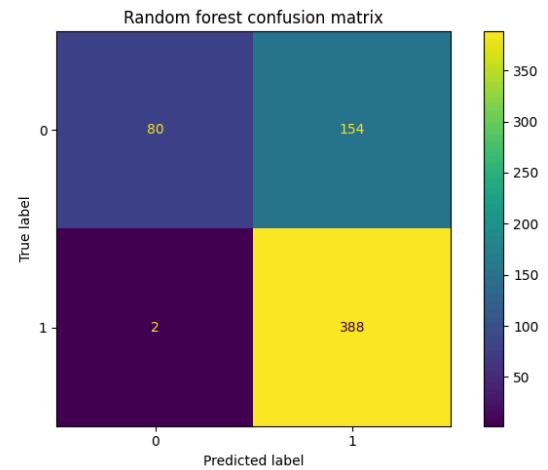
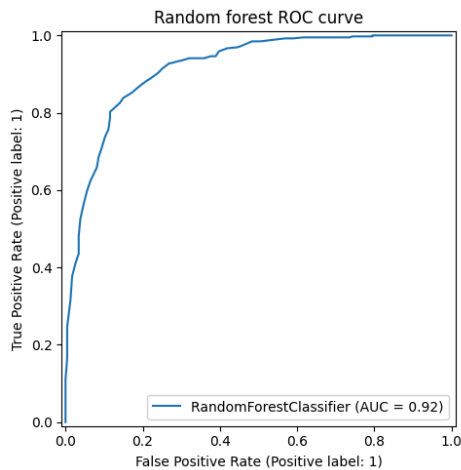
ConfusionMatrixDisplay.from_estimator(
    rf_model2, X_split2_test, y_split2_test, ax=axes[1])
```

```

axes[1].set_title('Random forest confusion matrix')

plt.tight_layout()
plt.savefig('../outputs/figures/RF_evaluation.png', dpi=150)
plt.show()
print("Random Forest plots saved.")

```



Random Forest plots saved.

```

In [23]: from sklearn.neural_network import MLPClassifier

print("MLP is ready.")

```

MLP is ready.

```

In [24]: # MLP - Split 1 (Simple Train/Test)
mlp_model = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    max_iter=300,
    random_state=42
)
mlp_model.fit(X_split1_train, y_split1_train)

y_pred_mlp = mlp_model.predict(X_split1_test)
y_prob_mlp = mlp_model.predict_proba(X_split1_test)[:,1]

print("MLP on split 1")
print(f" Accuracy : {accuracy_score(y_split1_test, y_pred_mlp):.4f}")
print(f" Precision : {precision_score(y_split1_test, y_pred_mlp):.4f}")
print(f" Recall : {recall_score(y_split1_test, y_pred_mlp):.4f}")
print(f" F1 : {f1_score(y_split1_test, y_pred_mlp):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split1_test, y_prob_mlp):.4f}")

```

MLP on split 1

Accuracy : 0.9741
Precision : 0.9845
Recall : 0.9807
F1 : 0.9826
ROC-AUC : 0.9961

```

In [25]: # MLP - Split 2 (Train/Val/Test)
mlp_model2 = MLPClassifier(
    hidden_layer_sizes=(256, 128),

```

```

        max_iter=300,
        random_state=42
    )
mlp_model2.fit(X_split2_train, y_split2_train)

y_pred_mlp2 = mlp_model2.predict(X_split2_test)
y_prob_mlp2 = mlp_model2.predict_proba(X_split2_test)[:,1]

print("MLP on split 2")
print(f" Accuracy : {accuracy_score(y_split2_test, y_pred_mlp2):.4f}")
print(f" Precision : {precision_score(y_split2_test, y_pred_mlp2):.4f}")
print(f" Recall : {recall_score(y_split2_test, y_pred_mlp2):.4f}")
print(f" F1 : {f1_score(y_split2_test, y_pred_mlp2):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_split2_test, y_prob_mlp2):.4f}")

```

```

MLP on split 2
Accuracy : 0.7788
Precision : 0.7432
Recall : 0.9872
F1 : 0.8480
ROC-AUC : 0.9039

```

```

In [26]: # MLP - Split 3 (K-Fold CV)
mlp_model3 = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    max_iter=300,
    random_state=42
)

cv_results_mlp = cross_validate(
    mlp_model3, X_train_pca, y_train,
    cv=skf,
    scoring=['accuracy', 'f1', 'roc_auc'],
    return_train_score=True
)

print("MLP with 5-fold cross-validation")
print(f" Accuracy : {cv_results_mlp['test_accuracy'].mean():.4f}")
print(f" F1 : {cv_results_mlp['test_f1'].mean():.4f}")
print(f" ROC-AUC : {cv_results_mlp['test_roc_auc'].mean():.4f}")

```

```

MLP with 5-fold cross-validation
Accuracy : 0.9643
F1 : 0.9760
ROC-AUC : 0.9930

```

```

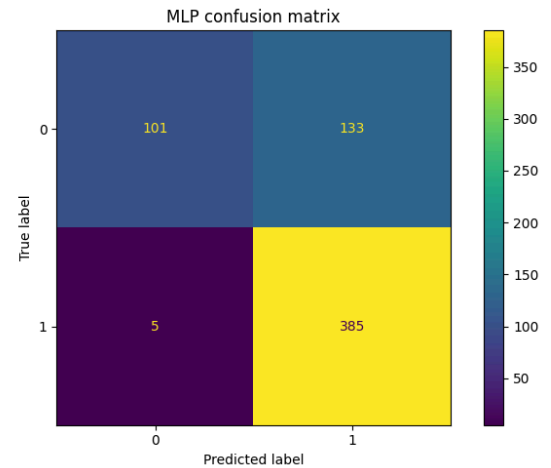
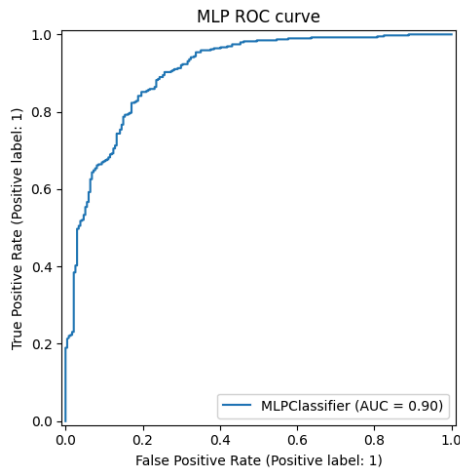
In [27]: # ROC Curve + Confusion Matrix for MLP
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

RocCurveDisplay.from_estimator(
    mlp_model2, X_split2_test, y_split2_test, ax=axes[0])
axes[0].set_title('MLP ROC curve')

ConfusionMatrixDisplay.from_estimator(
    mlp_model2, X_split2_test, y_split2_test, ax=axes[1])
axes[1].set_title('MLP confusion matrix')

```

```
plt.tight_layout()
plt.savefig('../outputs/figures/MLP_evaluation.png', dpi=150)
plt.show()
print("MLP plots saved.")
```



MLP plots saved.

```
In [28]: # Test current model on a real image from test set (in-memory pipeline)
import numpy as np
from PIL import Image
from sklearn.pipeline import Pipeline
import os

pipeline = Pipeline([
    ('scaler', scaler),
    ('pca', pca),
    ('classifier', mlp_model2)
])
print('In-memory pipeline ready!')

# Pick one NORMAL image from test set
normal_folder = '../data/test/NORMAL'
normal_files = os.listdir(normal_folder)
test_image_path = os.path.join(normal_folder, normal_files[0])

# Preprocess it
img = Image.open(test_image_path).convert('L')
img = img.resize((128, 128))
X_new = np.array(img).flatten() / 255.0
X_new = X_new.reshape(1, -1)

# Predict
result = pipeline.predict(X_new)[0]
probability = pipeline.predict_proba(X_new)[0]

print(f"\nImage      : {normal_files[0]}")
print('Actual      : NORMAL')
print(f'Predicted: {'PNEUMONIA' if result == 1 else 'NORMAL'}')
print(f'Confidence Normal      : {probability[0]:.2%}')
print(f'Confidence Pneumonia  : {probability[1]:.2%}')
print(f"\nCorrect      : {result == 0}")
```

In-memory pipeline ready!

Image : IM-0031-0001.jpeg
Actual : NORMAL
Predicted: NORMAL
Confidence Normal : 99.33%
Confidence Pneumonia : 0.67%

Correct : True

```
In [29]: # Pick one PNEUMONIA image from test set
pneumonia_folder = '../data/test/PNEUMONIA'
pneumonia_files = os.listdir(pneumonia_folder)
test_image_path2 = os.path.join(pneumonia_folder, pneumonia_files[0])

# Preprocess it
img2 = Image.open(test_image_path2).convert('L')
img2 = img2.resize((128, 128))
X_new2 = np.array(img2).flatten() / 255.0
X_new2 = X_new2.reshape(1, -1)

# Predict
result2 = pipeline.predict(X_new2)[0]
probability2 = pipeline.predict_proba(X_new2)[0]

print(f"Image : {pneumonia_files[0]}")
print(f"Actual : PNEUMONIA")
print(f"Predicted: {'PNEUMONIA' if result2 == 1 else 'NORMAL'}")
print(f"Confidence Normal : {probability2[0]:.2%}")
print(f"Confidence Pneumonia : {probability2[1]:.2%}")
print(f"\nCorrect : {result2 == 1}")
```

Image : person147_bacteria_706.jpeg
Actual : PNEUMONIA
Predicted: PNEUMONIA
Confidence Normal : 0.00%
Confidence Pneumonia : 100.00%

Correct : True

```
In [30]: import random

# Test on 5 NORMAL + 5 PNEUMONIA images
correct = 0
total = 0
results = []

for label, folder, true_val in [
    ('NORMAL', '../data/test/NORMAL', 0),
    ('PNEUMONIA', '../data/test/PNEUMONIA', 1)
]:
    files = os.listdir(folder)
    samples = random.sample(files, 5)

    for fname in samples:
        try:
```

```

img = Image.open(os.path.join(folder, fname)).convert('L')
img = img.resize((128, 128))
X = np.array(img).flatten() / 255.0
X = X.reshape(1, -1)

pred = pipeline.predict(X)[0]
prob = pipeline.predict_proba(X)[0]

is_correct = pred == true_val
correct += is_correct
total += 1

results.append({
    'file': fname,
    'actual': label,
    'predicted': 'PNEUMONIA' if pred == 1 else 'NORMAL',
    'correct': '✓' if is_correct else '✗',
    'confidence': f"{max(prob):.2%}"
})
except:
    pass

# Print results table
print(f"{'File':<40} {'Actual':<12} {'Predicted':<12} {'Correct':<8} {'Confidence':<12}")
print("-" * 90)
for r in results:
    print(f"{r['file']:<40} {r['actual']:<12} {r['predicted']:<12} {r['correct']:<8} {r['confidence']:<12}")

print(f"\nTotal Correct : {correct}/{total}")
print(f"Accuracy       : {correct/total:.2%}")

```

File Confidence	Actual	Predicted	Correct

IM-0001-0001.jpeg 89.76%	NORMAL	NORMAL	✓
NORMAL2-IM-0081-0001.jpeg 99.64%	NORMAL	NORMAL	✓
NORMAL2-IM-0380-0001.jpeg 99.53%	NORMAL	PNEUMONIA	✗
NORMAL2-IM-0051-0001.jpeg 99.96%	NORMAL	NORMAL	✓
NORMAL2-IM-0246-0001.jpeg 100.00%	NORMAL	PNEUMONIA	✗
person81_bacteria_395.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person103_bacteria_490.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person138_bacteria_658.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person62_virus_119.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person142_bacteria_683.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
Total Correct : 8/10			
Accuracy : 80.00%			

```
In [31]: # Retrain MLP with balanced class weights
mlp_balanced = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    max_iter=300,
    random_state=42
)

# We handle balance manually using sample weights
from sklearn.utils.class_weight import compute_sample_weight

sample_weights = compute_sample_weight('balanced', y_split2_train)

mlp_balanced.fit(X_split2_train, y_split2_train,
                  sample_weight=sample_weights)

# Test on same 10 images
correct = 0
total = 0
results = []

for label, folder, true_val in [
    ('NORMAL', '../data/test/NORMAL', 0),
    ('PNEUMONIA', '../data/test/PNEUMONIA', 1)
]:
    files = os.listdir(folder)
    samples = random.sample(files, 5)

    for fname in samples:
```

```

try:
    img = Image.open(os.path.join(folder, fname)).convert('L')
    img = img.resize((128, 128))
    X = np.array(img).flatten() / 255.0
    X = X.reshape(1, -1)
    X_scaled = scaler.transform(X)
    X_pca_new = pca.transform(X_scaled)

    pred = mlp_balanced.predict(X_pca_new)[0]
    prob = mlp_balanced.predict_proba(X_pca_new)[0]

    is_correct = pred == true_val
    correct += is_correct
    total += 1

    results.append({
        'file': fname,
        'actual': label,
        'predicted': 'PNEUMONIA' if pred == 1 else 'NORMAL',
        'correct': '✅' if is_correct else '❌',
        'confidence': f"{max(prob):.2%}"
    })
except:
    pass

print(f"{'File':<40} {'Actual':<12} {'Predicted':<12} {'Correct':<8} {'Confidence':<12}")
print("-" * 90)
for r in results:
    print(f"{r['file']:<40} {r['actual']:<12} {r['predicted']:<12} {r['correct']:<8} {r['confidence']:<12}")

print(f"\nTotal Correct : {correct}/{total}")
print(f"Accuracy : {correct/total:.2%}")

```

File Confidence	Actual	Predicted	Correct

NORMAL2-IM-0290-0001.jpeg 100.00%	NORMAL	PNEUMONIA	✗
NORMAL2-IM-0323-0001.jpeg 99.98%	NORMAL	NORMAL	✓
NORMAL2-IM-0309-0001.jpeg 100.00%	NORMAL	NORMAL	✓
NORMAL2-IM-0277-0001.jpeg 98.27%	NORMAL	PNEUMONIA	✗
IM-0016-0001.jpeg 98.26%	NORMAL	NORMAL	✓
person147_bacteria_707.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person111_bacteria_537.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person67_virus_126.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
person60_virus_117.jpeg 86.50%	PNEUMONIA	NORMAL	✗
person147_bacteria_706.jpeg 100.00%	PNEUMONIA	PNEUMONIA	✓
Total Correct : 7/10			
Accuracy : 70.00%			

```
In [32]: import random

# Test on 50 NORMAL + 50 PNEUMONIA = 100 random images
correct = 0
total = 0
results = []

normal_correct = 0
pneumonia_correct = 0

for label, folder, true_val in [
    ('NORMAL', '../data/test/NORMAL', 0),
    ('PNEUMONIA', '../data/test/PNEUMONIA', 1)
]:
    files = os.listdir(folder)
    samples = random.sample(files, 50)

    for fname in samples:
        try:
            img = Image.open(os.path.join(folder, fname)).convert('L')
            img = img.resize((128, 128))
            X = np.array(img).flatten() / 255.0
            X = X.reshape(1, -1)
            X_scaled = scaler.transform(X)
            X_pca_new = pca.transform(X_scaled)

            pred = mlp_balanced.predict(X_pca_new)[0]
            prob = mlp_balanced.predict_proba(X_pca_new)[0]
```

```

        is_correct = pred == true_val
        correct += is_correct
        total += 1

        if label == 'NORMAL' and is_correct:
            normal_correct += 1
        if label == 'PNEUMONIA' and is_correct:
            pneumonia_correct += 1

        results.append({
            'actual': label,
            'predicted': 'PNEUMONIA' if pred == 1 else 'NORMAL',
            'correct': '✓' if is_correct else '✗',
            'confidence': f"{max(prob):.2%}"
        })
    except:
        pass

# Print summary
print("=" * 50)
print("TEST RESULTS – 100 RANDOM IMAGES")
print("=" * 50)
print(f"\nNORMAL    correct : {normal_correct}/50")
print(f"PNEUMONIA correct : {pneumonia_correct}/50")
print(f"\nTotal Correct      : {correct}/100")
print(f"Overall Accuracy    : {correct/total:.2%}")

# Count errors
normal_wrong = 50 - normal_correct
pneumonia_wrong = 50 - pneumonia_correct
print(f"\nNORMAL    wrong    : {normal_wrong}/50")
print(f"PNEUMONIA wrong    : {pneumonia_wrong}/50")

```

```

=====
TEST RESULTS – 100 RANDOM IMAGES
=====

```

```

NORMAL    correct : 26/50
PNEUMONIA correct : 50/50

```

```

Total Correct      : 76/100
Overall Accuracy    : 76.00%

```

```

NORMAL    wrong    : 24/50
PNEUMONIA wrong    : 0/50

```

```

In [33]: # Way 1 – Load balanced dataset with rotation for NORMAL only
def load_dataset_balanced(path, size=(128, 128)):
    X, y = [], []
    for label, val in [('NORMAL', 0), ('PNEUMONIA', 1)]:
        folder = os.path.join(path, label)
        files = [f for f in os.listdir(folder)
                  if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        for fname in files:
            try:

```

```

        img = Image.open(os.path.join(folder, fname)).convert('L')
        img = img.resize(size)
        X.append(np.array(img).flatten() / 255.0)
        y.append(val)

    # Only rotate NORMAL to balance dataset
    if label == 'NORMAL':
        for angle in [10, -10, 20, -20]:
            img_r = img.rotate(angle)
            X.append(np.array(img_r).flatten() / 255.0)
            y.append(val)

    except:
        pass

    return np.array(X), np.array(y)

print("Loading balanced dataset with rotation...")
X_train_bal, y_train_bal = load_dataset_balanced(DATASET1)

print(f"\nBefore rotation:")
print(f"  NORMAL      : 1341")
print(f"  PNEUMONIA   : 3875")
print(f"\nAfter rotation:")
print(f"  NORMAL      : {(y_train_bal==0).sum()}")
print(f"  PNEUMONIA   : {(y_train_bal==1).sum()}")
print(f"  Total       : {len(y_train_bal)}")

```

Loading balanced dataset with rotation...

Before rotation:

```

NORMAL      : 1341
PNEUMONIA   : 3875

```

After rotation:

```

NORMAL      : 6705
PNEUMONIA   : 3875
Total       : 10580

```

```

In [34]: # Way 2 – Scale and PCA on balanced data then retrain MLP
print("Applying scaler and PCA...")
X_train_bal_scaled = scaler.transform(X_train_bal)
X_train_bal_pca     = pca.transform(X_train_bal_scaled)

print("Retraining MLP on balanced data...")
mlp_final = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    max_iter=300,
    random_state=42
)
mlp_final.fit(X_train_bal_pca, y_train_bal)
print("Done! MLP retrained on balanced dataset.")

```

Applying scaler and PCA...

Retraining MLP on balanced data...

Done! MLP retrained on balanced dataset.

```

In [35]: # Way 3 – Test on 100 random images
correct = 0
normal_correct = 0

```

```

pneumonia_correct = 0

for label, folder, true_val in [
    ('NORMAL', '../data/test/NORMAL', 0),
    ('PNEUMONIA', '../data/test/PNEUMONIA', 1)
]:
    files = os.listdir(folder)
    samples = random.sample(files, 50)

    for fname in samples:
        try:
            img = Image.open(os.path.join(folder, fname)).convert('L')
            img = img.resize((128, 128))
            X = np.array(img).flatten() / 255.0
            X = X.reshape(1, -1)
            X_scaled = scaler.transform(X)
            X_pca_new = pca.transform(X_scaled)

            pred = mlp_final.predict(X_pca_new)[0]
            is_correct = pred == true_val
            correct += is_correct

            if label == 'NORMAL' and is_correct:
                normal_correct += 1
            if label == 'PNEUMONIA' and is_correct:
                pneumonia_correct += 1
        except:
            pass

print("=" * 50)
print("NEW RESULTS - BALANCED + ROTATION")
print("=" * 50)
print(f"\nNORMAL    correct : {normal_correct}/50")
print(f"PNEUMONIA correct : {pneumonia_correct}/50")
print(f"\nTotal Correct    : {correct}/100")
print(f"Overall Accuracy  : {correct/100:.2%}")
print(f"\nNORMAL    wrong    : {50-normal_correct}/50")
print(f"PNEUMONIA wrong    : {50-pneumonia_correct}/50")

```

```

=====
NEW RESULTS - BALANCED + ROTATION
=====

```

```

NORMAL    correct : 16/50
PNEUMONIA correct : 49/50

```

```

Total Correct    : 65/100
Overall Accuracy  : 65.00%

```

```

NORMAL    wrong    : 34/50
PNEUMONIA wrong    : 1/50

```

```

In [36]: # Balance perfectly - match PNEUMONIA count
def load_dataset_perfect(path, size=(128, 128)):
    X, y = [], []
    for label, val in [('NORMAL', 0), ('PNEUMONIA', 1)]:

```

```

        folder = os.path.join(path, label)
        files = [f for f in os.listdir(folder)
                  if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        for fname in files:
            try:
                img = Image.open(os.path.join(folder, fname)).convert('L')
                img = img.resize(size)
                X.append(np.array(img).flatten() / 255.0)
                y.append(val)

                # Only rotate NORMAL - 2 rotations only
                if label == 'NORMAL':
                    for angle in [15, -15]:
                        img_r = img.rotate(angle)
                        X.append(np.array(img_r).flatten() / 255.0)
                        y.append(val)

            except:
                pass
        return np.array(X), np.array(y)

print("Loading perfectly balanced dataset...")
X_train_perf, y_train_perf = load_dataset_perfect(DATASET1)

print(f"NORMAL      : {(y_train_perf==0).sum()}")
print(f"PNEUMONIA   : {(y_train_perf==1).sum()}")
print(f"Total        : {len(y_train_perf)}")

```

```

Loading perfectly balanced dataset...
NORMAL      : 4023
PNEUMONIA   : 3875
Total       : 7898

```

```

In [37]: # Retrain on perfectly balanced data
print("Applying scaler and PCA...")
X_train_perf_scaled = scaler.transform(X_train_perf)
X_train_perf_pca    = pca.transform(X_train_perf_scaled)

print("Retraining MLP...")
mlp_perfect = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    max_iter=300,
    random_state=42
)
mlp_perfect.fit(X_train_perf_pca, y_train_perf)
print("Done!")

```

```

Applying scaler and PCA...
Retraining MLP...
Done!

```

```

In [38]: # Test on 100 random images
correct = 0
normal_correct = 0
pneumonia_correct = 0

for label, folder, true_val in [
    ('NORMAL',    '../data/test/NORMAL',    0),

```

```

('PNEUMONIA', '../data/test/PNEUMONIA', 1)
]:
files = os.listdir(folder)
samples = random.sample(files, 50)

for fname in samples:
    try:
        img = Image.open(os.path.join(folder, fname)).convert('L')
        img = img.resize((128, 128))
        X = np.array(img).flatten() / 255.0
        X = X.reshape(1, -1)
        X_scaled = scaler.transform(X)
        X_pca_new = pca.transform(X_scaled)

        pred = mlp_perfect.predict(X_pca_new)[0]
        is_correct = pred == true_val
        correct += is_correct

        if label == 'NORMAL' and is_correct:
            normal_correct += 1
        if label == 'PNEUMONIA' and is_correct:
            pneumonia_correct += 1
    except:
        pass

print("=" * 50)
print("PERFECTLY BALANCED RESULTS")
print("=" * 50)
print(f"\nNORMAL      correct : {normal_correct}/50")
print(f"PNEUMONIA correct : {pneumonia_correct}/50")
print(f"\nTotal Correct      : {correct}/100")
print(f"Overall Accuracy   : {correct/100:.2%}")
print(f"\nNORMAL      wrong    : {50-normal_correct}/50")
print(f"PNEUMONIA wrong    : {50-pneumonia_correct}/50")

```

```

=====
PERFECTLY BALANCED RESULTS
=====

```

```

NORMAL      correct : 22/50
PNEUMONIA correct : 50/50

```

```

Total Correct      : 72/100
Overall Accuracy   : 72.00%

```

```

NORMAL      wrong    : 28/50
PNEUMONIA wrong    : 0/50

```

In []:

```

In [39]: from sklearn.svm import SVC
         from sklearn.metrics import accuracy_score, precision_score, recall_score, f

         svm_model = SVC(probability=True, random_state=42)
         svm_model.fit(X_train_pca, y_train)

```

```

y_pred_svm = svm_model.predict(X_test_pca)
y_prob_svm = svm_model.predict_proba(X_test_pca)[:, 1]

print('SVM baseline results')
print(f" Accuracy : {accuracy_score(y_test, y_pred_svm):.4f}")
print(f" Precision : {precision_score(y_test, y_pred_svm):.4f}")
print(f" Recall : {recall_score(y_test, y_pred_svm):.4f}")
print(f" F1 : {f1_score(y_test, y_pred_svm):.4f}")
print(f" ROC-AUC : {roc_auc_score(y_test, y_prob_svm):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
RocCurveDisplay.from_estimator(svm_model, X_test_pca, y_test, ax=axes[0])
axes[0].set_title('SVM ROC curve')
ConfusionMatrixDisplay.from_estimator(svm_model, X_test_pca, y_test, ax=axes[1])
axes[1].set_title('SVM confusion matrix')
plt.tight_layout()
plt.savefig('../outputs/figures/SVM_evaluation.png', dpi=150)
plt.show()

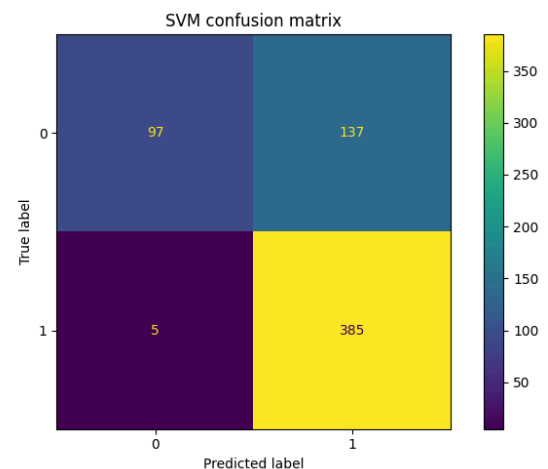
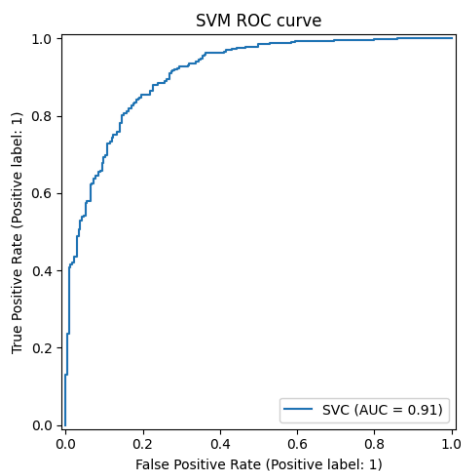
```

SVM baseline results

```

Accuracy : 0.7724
Precision : 0.7375
Recall : 0.9872
F1 : 0.8443
ROC-AUC : 0.9093

```



```

In [40]: k_values = [3, 5, 7, 9, 11]
         k_scan = []

```

```

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_pca, y_train)
    y_pred_k = model.predict(X_test_pca)
    y_prob_k = model.predict_proba(X_test_pca)[:, 1]
    k_scan.append({
        'k': k,
        'F1': f1_score(y_test, y_pred_k),
        'ROC-AUC': roc_auc_score(y_test, y_prob_k),
    })

```

```

k_scan_df = pd.DataFrame(k_scan).sort_values(['ROC-AUC', 'F1'], ascending=False)
best_k = int(k_scan_df.iloc[0]['k'])

```

```

display(k_scan_df)
print(f"Best k from the quick scan: {best_k}")

knn_model = KNeighborsClassifier(n_neighbors=best_k)
knn_model.fit(X_train_pca, y_train)

y_pred_knn = knn_model.predict(X_test_pca)
y_prob_knn = knn_model.predict_proba(X_test_pca)[:, 1]
print(f" Accuracy : {accuracy_score(y_test, y_pred_knn):.4f}")
print(f" Precision : {precision_score(y_test, y_pred_knn):.4f}")
print(f" Recall    : {recall_score(y_test, y_pred_knn):.4f}")
print(f" F1        : {f1_score(y_test, y_pred_knn):.4f}")
print(f" ROC-AUC    : {roc_auc_score(y_test, y_prob_knn):.4f}")

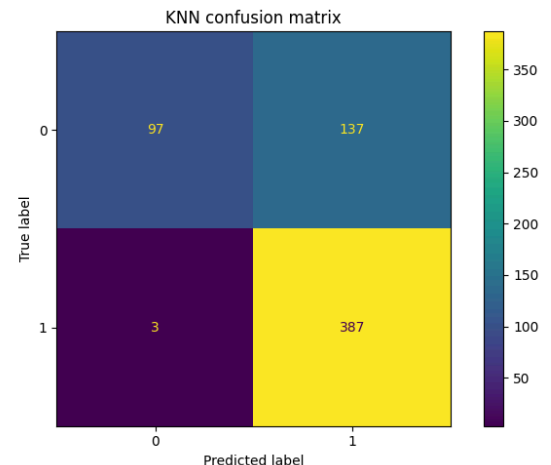
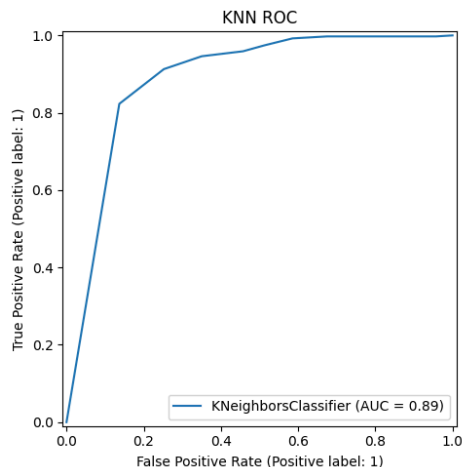
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
RocCurveDisplay.from_estimator(knn_model, X_test_pca, y_test, ax=axes[0])
axes[0].set_title('KNN ROC')
ConfusionMatrixDisplay.from_estimator(knn_model, X_test_pca, y_test, ax=axes[1])
axes[1].set_title('KNN confusion matrix')
plt.tight_layout()
plt.savefig('../outputs/figures/KNN_evaluation.png', dpi=150)
plt.show()

```

	k	F1	ROC-AUC
4	11	0.846827	0.887990
3	9	0.844057	0.874332
2	7	0.845902	0.862366
1	5	0.846827	0.833394
0	3	0.853333	0.804865

Best k from the quick scan: 11

Accuracy : 0.7756
 Precision : 0.7385
 Recall : 0.9923
 F1 : 0.8468
 ROC-AUC : 0.8880



```

In [41]: # Tune on Dataset 2 (validation) using a predefined split
X_tune = np.vstack([X_train_pca, X_val_pca])
y_tune = np.concatenate([y_train, y_val])

test_fold = np.concatenate([
    np.full(len(X_train_pca), -1),
    np.zeros(len(X_val_pca), dtype=int),
])
val_split = PredefinedSplit(test_fold=test_fold)

def tune_model(estimator, param_grid, model_name):
    search = GridSearchCV(
        estimator=estimator,
        param_grid=param_grid,
        scoring='roc_auc',
        cv=val_split,
        n_jobs=-1,
        refit=True,
    )
    search.fit(X_tune, y_tune)
    print(f"{model_name} best params: {search.best_params_}")
    return search.best_estimator_, search.best_params_

lr_best_model, lr_best_params = tune_model(
    LogisticRegression(max_iter=1000, random_state=42, solver='liblinear'),
    {'C': [0.01, 0.1, 1, 10], 'penalty': ['l1', 'l2']},
    'Logistic Regression',
)

rf_best_model, rf_best_params = tune_model(
    RandomForestClassifier(random_state=42),
    {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20]},
    'Random Forest',
)

svm_best_model, svm_best_params = tune_model(
    SVC(probability=True, random_state=42),
    {'C': [0.1, 1, 10], 'kernel': ['rbf', 'linear'], 'gamma': ['scale', 'auto']},
    'SVM',
)

knn_best_model, knn_best_params = tune_model(
    KNeighborsClassifier(),
    {'n_neighbors': [3, 5, 7, 9, 11], 'weights': ['uniform', 'distance']},
    'KNN',
)

```

```

Logistic Regression best params: {'C': 0.01, 'penalty': 'l2'}
Random Forest best params: {'max_depth': 10, 'n_estimators': 50}
SVM best params: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
KNN best params: {'n_neighbors': 7, 'weights': 'uniform'}

```

```

In [42]: def evaluate_for_table(model_name, person, model):
    y_pred = model.predict(X_test_pca)
    y_prob = model.predict_proba(X_test_pca)[:, 1]
    return {

```

```

        'Model': model_name,
        'Person': person,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1': f1_score(y_test, y_pred),
        'ROC-AUC': roc_auc_score(y_test, y_prob),
    }

comparison_rows = [
    evaluate_for_table('Logistic Regression', 'Jeet', lr_best_model),
    evaluate_for_table('Random Forest', 'Jeet', rf_best_model),
    evaluate_for_table('SVM', 'Teammate', svm_best_model),
    evaluate_for_table('KNN', 'Teammate', knn_best_model),
    evaluate_for_table('MLP neural net', 'Jeet', mlp_model2),
]

final_table = pd.DataFrame(comparison_rows)
final_table = final_table[['Model', 'Person', 'Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC']]
display(final_table.sort_values(['ROC-AUC', 'F1'], ascending=False))

```

	Model	Person	Accuracy	Precision	Recall	F1	ROC-AUC
1	Random Forest	Jeet	0.713141	0.687389	0.992308	0.812172	0.918639
2	SVM	Teammate	0.775641	0.740385	0.987179	0.846154	0.909237
4	MLP neural net	Jeet	0.778846	0.743243	0.987179	0.848018	0.903895
0	Logistic Regression	Jeet	0.777244	0.746562	0.974359	0.845384	0.884013
3	KNN	Teammate	0.774038	0.737143	0.992308	0.845902	0.862366

```

In [43]: import os
import joblib
from sklearn.pipeline import Pipeline

os.makedirs('../models', exist_ok=True)

model_lookup = {
    'Logistic Regression': lr_best_model,
    'Random Forest': rf_best_model,
    'SVM': svm_best_model,
    'KNN': knn_best_model,
    'MLP neural net': mlp_model2,
}

best_row = final_table.sort_values(['ROC-AUC', 'F1'], ascending=False).iloc[0]
best_model_name = best_row['Model']
best_model = model_lookup[best_model_name]

best_pipeline = Pipeline([
    ('scaler', scaler),
    ('pca', pca),

```

```
    ('classifier', best_model),  
])  
  
joblib.dump(best_pipeline, '../models/best_pipeline.joblib')  
print(f"Saved: ../models/best_pipeline.joblib ({best_model_name})")
```

Saved: ../models/best_pipeline.joblib (Random Forest)